
Generalization in diffusion models

MIT 6.S183 (IAP 2026): Lecture 6

Chris Scarvelis

January 16, 2026

So far...

Lecture 1
Introduction to
diffusion models

Lecture 3
Conditioning
and guidance

Lecture 5
Diffusion for
robotics

Lecture 2
Theoretical
perspectives

Lecture 4
Score distillation
and advanced
applications



Today...

***Generalization* in diffusion models.**



Questions for today.

- What does it mean for a diffusion model to *generalize*?
- *Do* diffusion models generalize?
- *Should* they generalize?
- *Why* do they generalize?

Defining generalization.

**What does it mean for a diffusion model to
generalize?**

Defining generalization.

Informally:

A diffusion model *generalizes* if it generates novel and realistic samples.

Defining generalization.



Realism is subjective.

Source: <https://civitai.com/models/139562/realvisxl-v50>

**We have metrics like *Fréchet inception distance* (FID),
but these are imperfect measures.**

Defining generalization.

Formally:

A diffusion model encodes a probability distribution p_θ .

We're typically interested in *sampling* from p_θ .

Defining generalization.

But we can also query p_θ to obtain the log-likelihood $\log p_\theta(z)$ of a sample z .

(Apply the continuous change of variables formula to the *probability flow ODE*.)

Defining generalization.

A diffusion model *generalizes* if

$$\mathbb{E}_{z \sim \mathcal{K}} [\log p_{\theta}(z)]$$

Average log-likelihood on
the *true* distribution

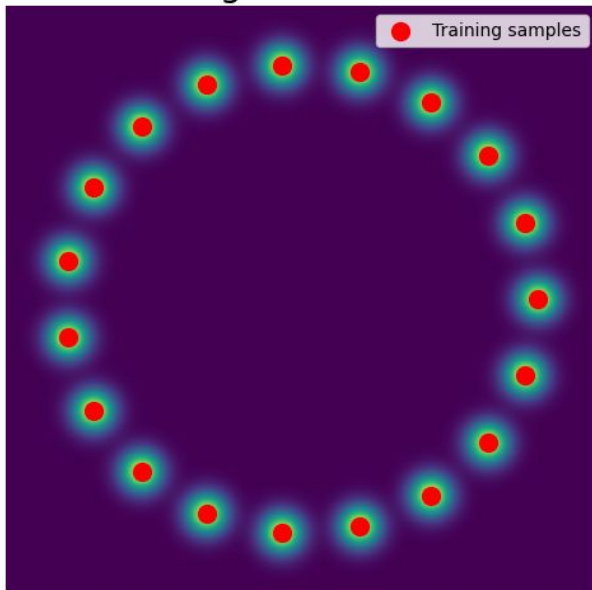
is large relative to the *empirical*

log-likelihood: $\frac{1}{n} \sum_{i=1}^n \log p_{\theta}(x_i)$

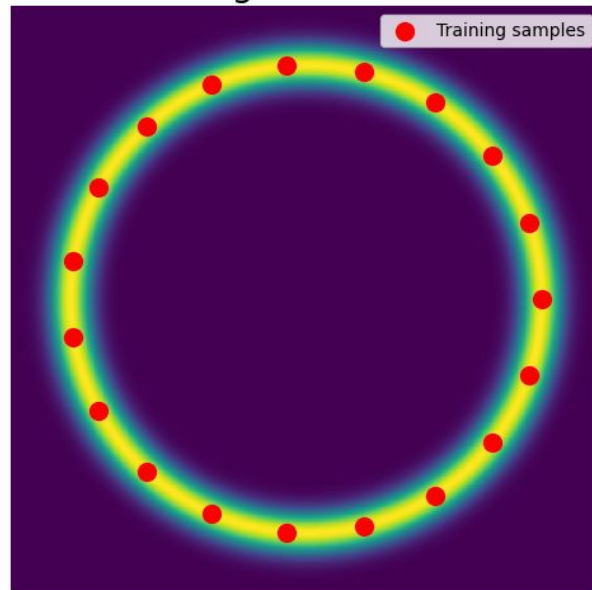
Average log-likelihood on
the *training samples*

Defining generalization.

Poor generalization



Good generalization



Caveat.

A NOTE ON THE EVALUATION OF GENERATIVE MODELS

Lucas Theis*

University of Tübingen
72072 Tübingen, Germany
lucas@bethgelab.org

Aäron van den Oord*†

Ghent University
9000 Ghent, Belgium
aaron.vandenoord@ugent.be

Matthias Bethge

University of Tübingen
72072 Tübingen, Germany
matthias@bethgelab.org

inspection can work well in low dimensions for assessing a model's fit to data. Unfortunately these intuitions can be misleading when the image dimensionality is high. A model can have poor log-likelihood and produce great samples, or have great log-likelihood and produce poor samples.



Do diffusion models generalize?



Do diffusion models generalize?

According to our informal definition, they should generate novel and realistic samples.

Do they generate novel samples?

Somepalli et al. (2022) find that at least 1.88% of Stable Diffusion samples replicate content from its training data.

Do they generate novel samples?

1.88% is quite a bit of content replication!

Enough to trigger copyright concerns.

Do they generate novel samples?

Legal implications:

UNITED STATES DISTRICT COURT NORTHERN DISTRICT OF CALIFORNIA SAN FRANCISCO DIVISION Sarah Andersen, an individual; Kelly McKernan, an individual; Karla Ortiz, an individual; H. Southworth pka Hawke Southworth, an individual; Grzegorz Rutkowski, an individual; Gregory Manchess, an individual; Gerald Brom, an individual; Jingna Zhang, an individual; Julia Kaye, an individual; Adam Ellis, an individual; Individual and Representative Plaintiffs, v. Stability AI Ltd., a UK corporation; Stability AI, Inc., a Delaware corporation; DeviantArt, Inc., a Delaware corporation; Midjourney, Inc., a Delaware corporation; Runway AI, Inc., a Delaware corporation; Defendants.	Case No. 3:23-cv-00201-WHO FIRST AMENDED COMPLAINT CLASS ACTION DEMAND FOR JURY TRIAL
--	--

prompts and between models. This strongly suggests that the Stable Diffusion XL, the Runway model, and the Midjourney Model store copies of protected expression after copying and ingesting training images.



Do they generate novel samples?

But on the other hand, ~98% of Stable Diffusion samples are novel.

Diffusion models *typically* generate new content.

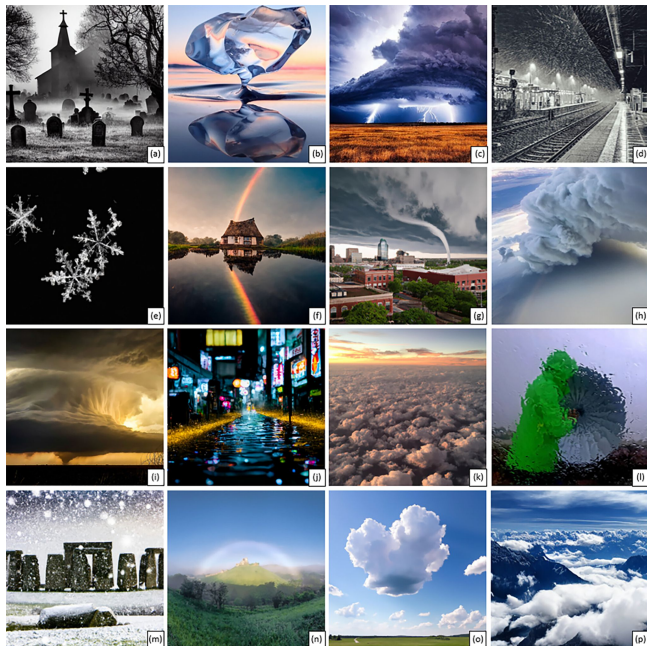
Do they generate realistic samples?

Are their samples *realistic*? You be the judge!



Source: <https://civitai.com/models/139562/realvisxl-v50>

Do they generate realistic samples?



Hunt (2022) proposes an informal Turing test for AI-generated images.



Formal generalization.

A diffusion model *generalizes* if

$$\mathbb{E}_{z \sim \mathcal{K}} [\log p_{\theta}(z)]$$

Average log-likelihood on
the *true* distribution

is large relative to the *empirical*

log-likelihood: $\frac{1}{n} \sum_{i=1}^n \log p_{\theta}(x_i)$

Average log-likelihood on
the *training* samples

In principle, we can approximate $\mathbb{E}_{z \sim \mathcal{K}} [\log p_{\theta}(z)]$
by averaging $\log p_{\theta}(z)$ over test samples $\mathbf{z}$.

(Caveat: Very noisy estimates in practice!)

Formal generalization.

“Maximum Likelihood Training of Score-Based Diffusion Models” reports test log-likelihoods:

Table 2: Negative log-likelihood (bits/dim) and sample quality (FID scores) on CIFAR-10 and ImageNet 32 × 32. Abbreviations: “NLL” for “negative log-likelihood”; “Uni. deq.” for “Uniform dequantization”; “Var. deq.” for “Variational dequantization”; “LW” for “likelihood weighting”; and “IS” for “importance sampling”. Bold indicates best result in the corresponding column. Shaded rows represent models trained with both likelihood weighting and importance sampling.

Model	SDE	CIFAR-10					ImageNet 32 × 32				
		Uni. deq.		Var. deq.		FID↓	Uni. deq.		Var. deq.		FID↓
		NLL↓	Bound↓	NLL↓	Bound↓		NLL↓	Bound↓	NLL↓	Bound↓	
Baseline	VP	3.16	3.28	3.04	3.14	3.98	3.90	3.96	3.84	3.91	8.34
Baseline + LW	VP	3.06	3.18	2.94	3.03	5.18	3.91	3.96	3.86	3.92	17.75
Baseline + LW + IS	VP	2.95	3.08	2.83	2.94	6.03	3.86	3.92	3.80	3.88	11.15
Deep	VP	3.13	3.25	3.01	3.10	3.09	3.89	3.95	3.84	3.90	8.40
Deep + LW	VP	3.06	3.17	2.93	3.02	7.88	3.91	3.96	3.86	3.92	17.73
Deep + LW + IS	VP	2.93	3.06	2.80	2.92	5.34	3.85	3.92	3.79	3.88	11.20
Baseline	subVP	2.99	3.09	2.88	2.98	3.20	3.87	3.92	3.82	3.88	8.71
Baseline + LW	subVP	2.97	3.07	2.86	2.96	7.33	3.87	3.92	3.82	3.88	12.99
Baseline + LW + IS	subVP	2.94	3.05	2.84	2.94	5.58	3.84	3.91	3.79	3.87	10.57
Deep	subVP	2.96	3.06	2.85	2.95	2.86	3.86	3.91	3.81	3.87	8.87
Deep + LW	subVP	2.95	3.05	2.85	2.94	6.57	3.88	3.93	3.83	3.88	16.55
Deep + LW + IS	subVP	2.90	3.02	2.81	2.90	5.40	3.82	3.90	3.76	3.86	10.18

But no empirical log-likelihoods for comparison!



Formal generalization.

Normalizing flows generalize well! (Nalisnick et al. 2022)

Data Set	Avg. Bits Per Dimension	Data Set	Avg. Bits Per Dimension	
<i>Glow Trained on FashionMNIST</i>		<i>Glow Trained on CIFAR-10</i>		
FashionMNIST-Train	2.902	CIFAR10-Train	3.386	Small generalization gap.
FashionMNIST-Test	2.958	CIFAR10-Test	3.464	
MNIST-Test	1.833	SVHN-Test	2.389	
<i>Glow Trained on MNIST</i>		<i>Glow Trained on SVHN</i>		
MNIST-Test	1.262	SVHN-Test	2.057	

Figure 1: *Testing Out-of-Distribution*. Log-likelihood (expressed in bits per dimension) calculated from Glow (Kingma & Dhariwal, 2018) on MNIST, FashionMNIST, SVHN, CIFAR-10.

Too well?

Data Set	Avg. Bits Per Dimension	Data Set	Avg. Bits Per Dimension
<i>Glow Trained on FashionMNIST</i>		<i>Glow Trained on CIFAR-10</i>	
FashionMNIST-Train	2.902	CIFAR10-Train	3.386
FashionMNIST-Test	2.958	CIFAR10-Test	3.464
MNIST-Test	1.833	SVHN-Test	2.389

***Better* log-likelihoods on out-of-distribution data!**

Kamkari et al. (2024): High likelihood in a region does not imply large probability mass if that region has low *volume*.

$$\text{Probability mass} \approx \text{Density} \times \text{Volume}$$

Diffusion models generalize.

So far, we've:

- *Defined* generalization
- **Seen that diffusion models generalize** (informally)

Should they generalize?

It's *surprising* that diffusion models generalize!

Should they generalize?

Diffusion models generate samples using a *denoiser* learned from training data.

Should they generalize?

We saw in Lecture 1 that for any *finite* training set, you can directly compute the ideal denoiser!

Recall: Lecture 1

The *ideal denoiser* is the minimizer of training loss, a function of data distribution $\mathcal{K}$ and noise level σ

$$\epsilon^* := \arg \min_{\epsilon_\theta} \mathbf{E}_{x_0, \sigma, \epsilon} \|\epsilon_\theta(x_0 + \sigma \epsilon, \sigma) - \epsilon\|^2$$

For finite $\mathcal{K}$, there is a closed-form solution:

$$\epsilon^*(x_\sigma, \sigma) = \frac{\sum_{x_0 \in \mathcal{K}} (x_\sigma - x_0) \exp(-\|x_\sigma - x_0\|^2 / 2\sigma^2)}{\sigma \sum_{x_0 \in \mathcal{K}} \exp(-\|x_\sigma - x_0\|^2 / 2\sigma^2)}$$

The ideal denoiser.

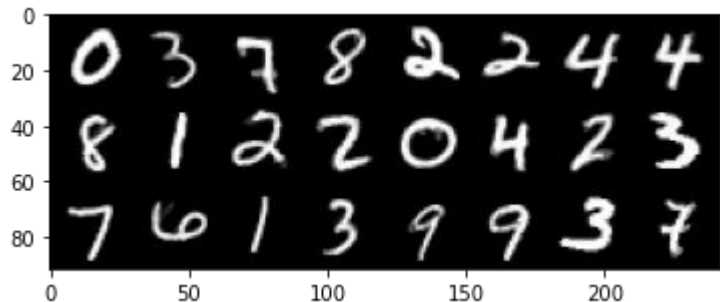
For finite $\mathcal{K}$, there is a closed-form solution:

$$\epsilon^*(x_\sigma, \sigma) = \frac{\sum_{x_0 \in \mathcal{K}} (x_\sigma - x_0) \exp(-\|x_\sigma - x_0\|^2 / 2\sigma^2)}{\sigma \sum_{x_0 \in \mathcal{K}} \exp(-\|x_\sigma - x_0\|^2 / 2\sigma^2)}$$

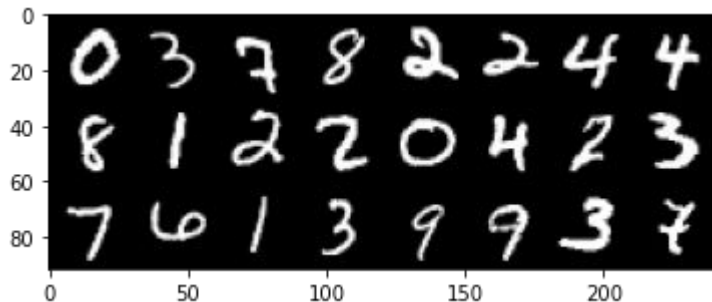
- **Easy to approximate** ([Scarvelis et al. 2023](#))
- **But: Using this denoiser in a diffusion model can *only* regurgitate training data!**

Diffusion models shouldn't generalize.

Optimal diffusion models **cannot** generate new samples.



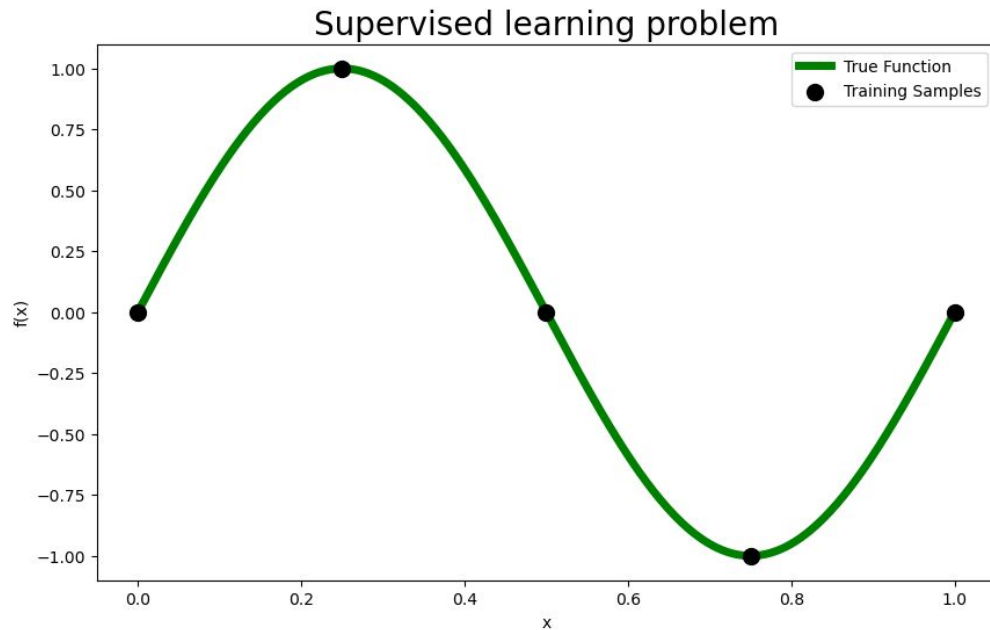
Model samples



Nearest neighbors in MNIST

How is this different from overfitting?

Supervised learning:
Given n training samples
 $(x_i, f(x_i))$, find f



How is this different from overfitting?

The function we're looking for *should* perfectly fit the training samples:

$$\min_{\hat{f}} \frac{1}{n} \sum_{i=1}^n L(f(x_i), \hat{f}(x_i))$$

(But also agree with the true f everywhere else.)

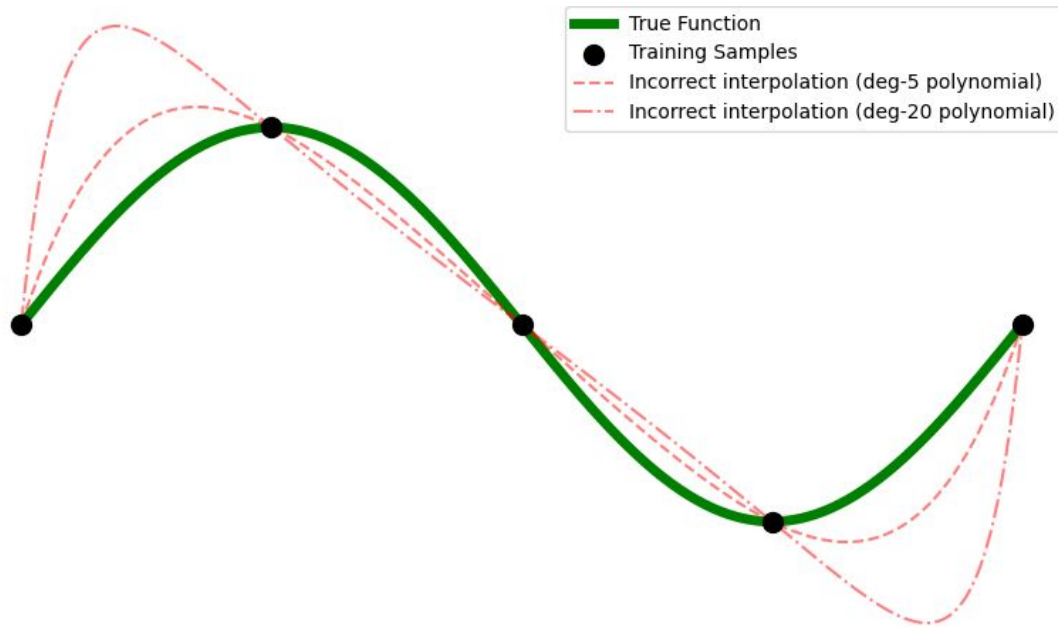
How is this different from overfitting?

Many solutions to:

$$\min_{\hat{f}} \frac{1}{n} \sum_{i=1}^n L(f(x_i), \hat{f}(x_i))$$

We hope to find the right one: The true f

How is this different from overfitting?



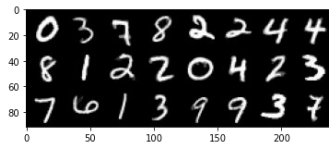
How is this different from overfitting?

For finite $\mathcal{K}$, there is a closed-form solution:

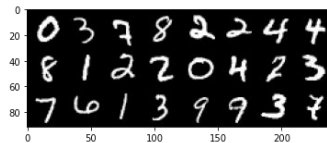
$$\epsilon^*(x_\sigma, \sigma) = \frac{\sum_{x_0 \in \mathcal{K}} (x_\sigma - x_0) \exp(-\|x_\sigma - x_0\|^2 / 2\sigma^2)}{\sigma \sum_{x_0 \in \mathcal{K}} \exp(-\|x_\sigma - x_0\|^2 / 2\sigma^2)}$$

But for diffusion models, there's just one solution to the training problem, and it's useless to us!

(...for finite training sets)



Model samples



Nearest neighbors in MNIST

How is this different from overfitting?

Generalization in diffusion models *only* happens if you *fail* to solve the training problem:

A *denoiser* $\epsilon_\theta : \mathbb{R}^n \times \mathbb{R}_+ \rightarrow \mathbb{R}^n$ is learned by minimizing

$$L(\theta) := \mathbf{E}_{x_0, \sigma, \epsilon} \|\epsilon_\theta(x_0 + \sigma\epsilon, \sigma) - \epsilon\|^2$$

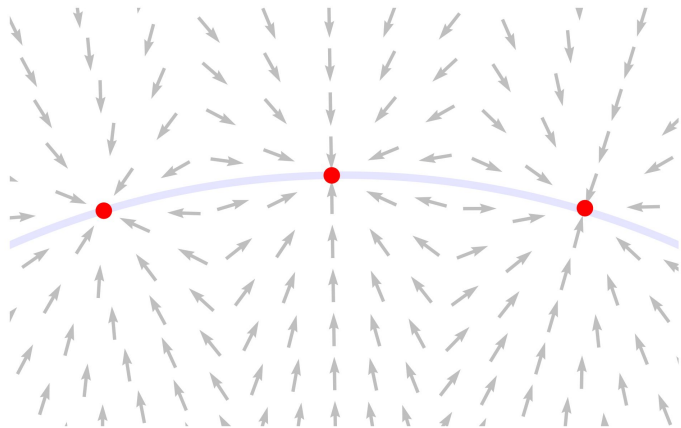
***Double descent* and *grokking* aren't sufficient explanations...**

***Why* do diffusion models generalize?**



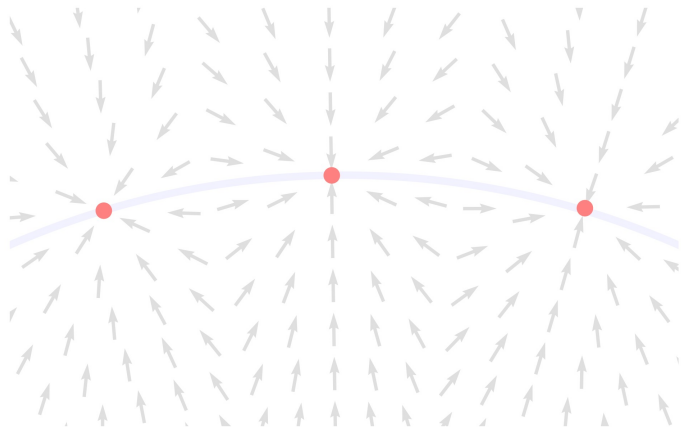
Why do diffusion models generalize?

Diffusion models **generalize** because we *don't* learn optimal models for arbitrary training sets.

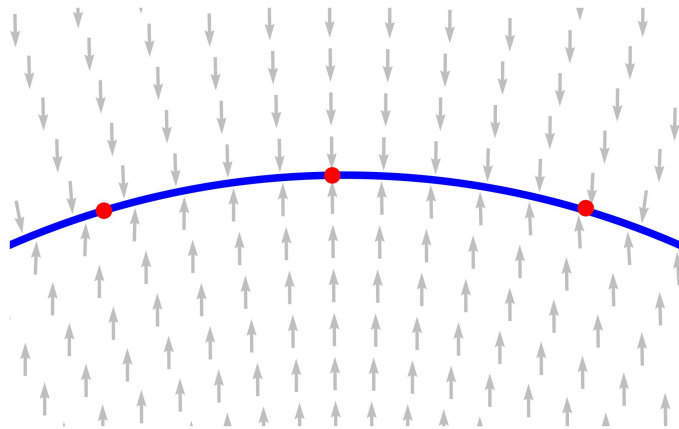


Why do diffusion models generalize?

Diffusion models **generalize** because we *don't* learn optimal models for arbitrary training sets.



We train neural networks via gradient-based methods on structured data, and we stop before convergence.



Why do diffusion models generalize?

Causes of **generalization**

Inductive biases

- Smoothness
([Scarvelis et al., 2025](#))
- Equivariance+locality
([Kamb and Ganguli, 2025](#))
- Geometry-adaptive harmonic bases
([Kadkhodaie et al. 2024](#))

Training procedure

- Early stopping
([Favero et al., 2025](#))
([Bonnaire et al., 2025](#))
- Stability
([Farghly et al., 2025a](#))

Data geometry

- Locality
([Lukoianov et al., 2025](#))
- Manifold structure?
([Farghly et al., 2025b](#))



Inductive biases: GAHBs

Published as a conference paper at ICLR 2024

GENERALIZATION IN DIFFUSION MODELS ARISES FROM GEOMETRY-ADAPTIVE HARMONIC REPRESENTATIONS

Zahra Kadkhodaie

Ctr. for Data Science, New York University
zk388@nyu.edu

Florentin Guth

Ctr. for Data Science, New York University
Flatiron Institute, Simons Foundation
florentin.guth@nyu.edu

Eero P. Simoncelli

New York University
Flatiron Institute, Simons Foundation
eero.simoncelli@nyu.edu

Stéphane Mallat

Collège de France
Flatiron Institute, Simons Foundation
stephane.mallat@ens.fr



Massachusetts Institute of Technology

6.S183 IAP 2026: A practical introduction to diffusion models
Slide 42

Local analysis

With ReLU activations, the denoiser is piecewise linear:

the denoiser mapping can be rewritten in terms of the Jacobian eigendecomposition as

$$f(y) = \nabla f(y) y = \sum_k \lambda_k(y) \langle y, e_k(y) \rangle e_k(y). \quad (6)$$

Noisy input y is indicated by an arrow pointing to y in the equation.

Linear combination of eigenvectors $e_k(y)$ is indicated by an arrow pointing to $e_k(y)$ in the equation.

Local analysis

the denoiser mapping can be rewritten in terms of the Jacobian eigendecomposition as

$$f(y) = \nabla f(y) y = \sum_k \lambda_k(y) \langle y, e_k(y) \rangle e_k(y). \quad (6)$$

Noisy input $\nearrow$

$\uparrow$
Linear combination of eigenvectors $e_k(y)$

The denoiser transforms the noisy input y into a linear combination of basis vectors $e_k(y)$

Local analysis

Q: So what? Any vector is a linear combination of basis vectors!

A: The training objective encourages *sparse* linear combinations.

$$f(y) = \nabla f(y) y = \sum_k \lambda_k(y) \langle y, e_k(y) \rangle e_k(y).$$

↑
Mostly ~0

Rewriting the training loss.

the denoiser mapping can be rewritten in terms of the Jacobian eigendecomposition as

$$f(y) = \nabla f(y) y = \sum_k \lambda_k(y) \langle y, e_k(y) \rangle e_k(y). \quad (6)$$

$$MSE(f) = \mathbb{E}_y \left[\underbrace{\|f(y) - y\|^2}_{\text{Accurately reconstruct noisy inputs}} + 2\sigma^2 \underbrace{\sum_{k=1}^n \lambda_k(y)}_{\text{Make } \lambda_k(y) \text{ small}} \right]$$

Sparse representations.

$$f(y) = \nabla f(y) y = \sum_k \lambda_k(y) \langle y, e_k(y) \rangle e_k(y).$$

↑
Mostly ~0

Faithfully reconstruct the noisy images...
...with *just a few* basis vectors.

Sparse representations aren't enough.

The basis vectors depend on the noisy inputs.

So this doesn't rule out memorization!

$$f(y) = \nabla f(y) y = \sum_k \lambda_k(y) \langle y, e_k(y) \rangle e_k(y).$$

↑
Mostly ~0

→

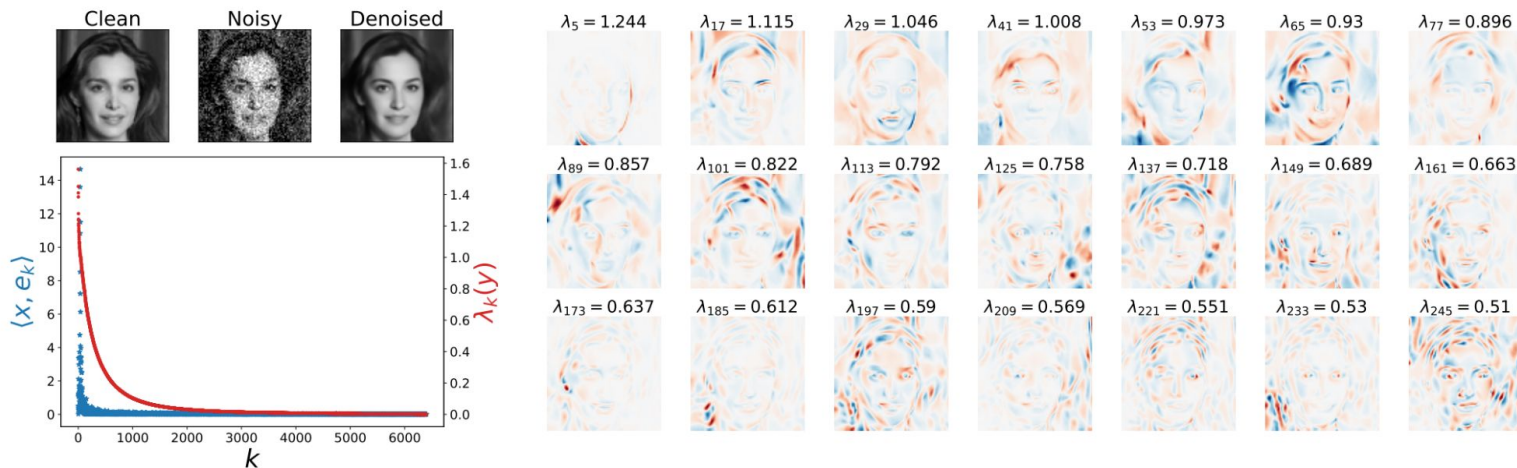
$e_1(y) = \text{Nearest training sample to } y$
 $\lambda_2 = \dots = \lambda_n = 0$

}

Sparse *memorizing* solutions are possible

Inductive biases: GAHBs

But in practice...

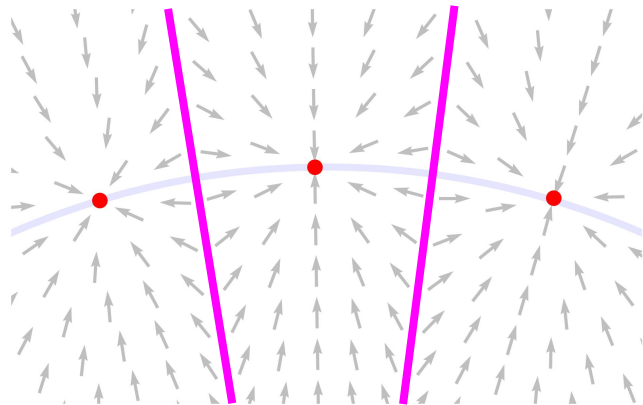


Inductive biases: GAHBs

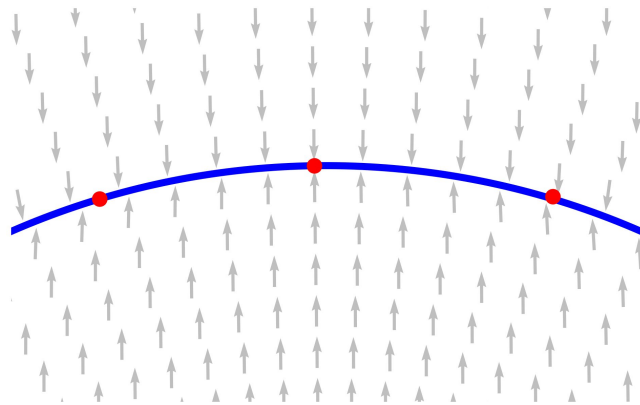
So in this story, neural denoisers:

- **Reconstruct noisy images as linear combinations of a few basis vectors.**
- **Basis vectors capture relevant geometric features without memorizing training data.**

Inductive biases: Smoothness



Optimal (memorizing) denoiser changes direction sharply near **Voronoi boundaries** of **training samples**.



(Generalizing) neural approximation remains smooth everywhere.

Inductive biases: Smoothness

A natural hypothesis:

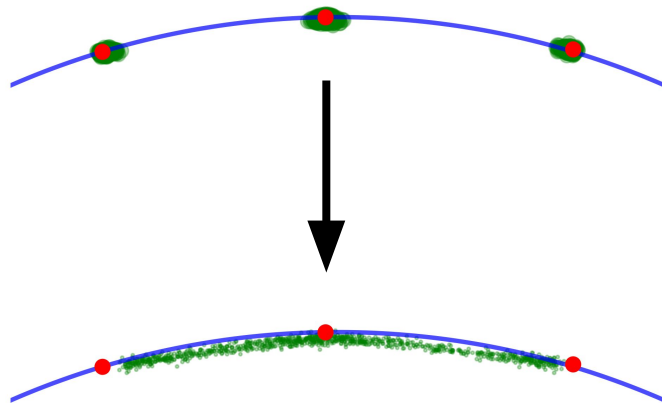
Neural diffusion models generalize because they learn smooth approximations to the optimal denoiser.

A known bias of neural networks! (Rahaman et al., 2019)

Inductive biases: Smoothness

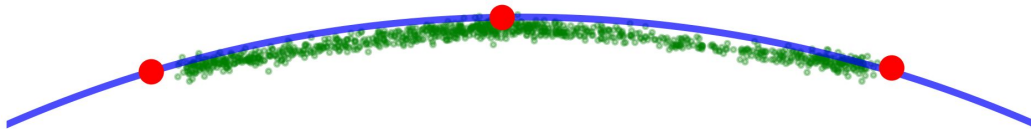
Samples from a *smoothed* optimal diffusion model lie on barycenters of nearby training samples.

(Scarvelis et al., 2025)



Inductive biases: Smoothness

Manifold hypothesis: Real-world data lives on low-dimensional “**surfaces**” in high-dimensional space.



With sufficiently dense **training data**, **samples** from the smoothed model lie near the **data manifold**!

Farghly et al. (2025) theoretically characterize the impact of the smoothing kernel, manifold curvature, and data dimension.

Inductive biases: ConvNets

Recent paper: “An analytic theory of creativity in convolutional diffusion models” ([Kamb & Ganguli, 2025](#))

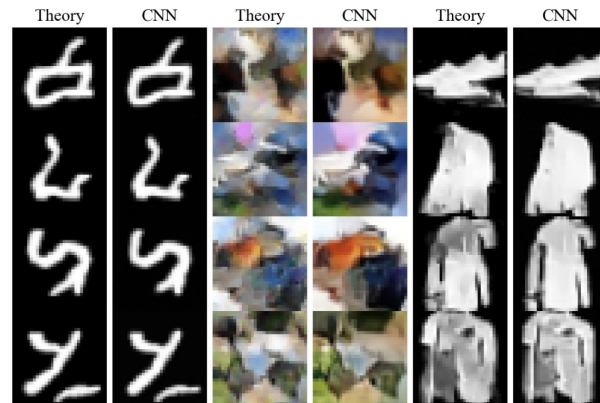
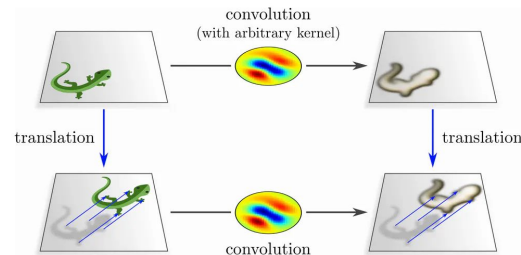
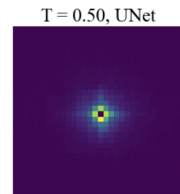


Figure 1. Our analytic theory (left columns) can accurately predict on a *case by case basis* the outputs of convolutional diffusion models (right columns), with U-Net or ResNet architectures trained on MNIST, CIFAR10 and FashionMNIST (left to right), even when these outputs are highly creative and far from the training data. See Fig. 5, App. C, Fig. 8 and Table 1, and App. D, Fig. 11 to Fig. 18 for many more successful theory-experiment comparisons.

Inductive biases: ConvNets

ConvNets have two notable inductive biases:

- **Locality**
 - Outputs of a ConvNet depend on their inputs only through a small receptive field of neighboring input pixels.
- **Equivariance**
 - Translating the input image also translates the ConvNet outputs.



Source: Maurice Weiler

Inductive biases: ConvNets

Can obtain an exact, closed-form solution to the denoising problem, *even* with locality and equivariance constraints.

Taken together, the ELS machine can creatively generate exponentially many novel images by mixing and matching local training set patches from any location and placing elements of them at any location in the generated image. We call this a *patch mosaic model of creativity*.

Inductive biases: ConvNets

Does a good job
predicting the outputs of a
ConvNet-based diffusion
model.



(b) Theory (left) vs. UNet (right)

But wait...

In this work, we present evidence that the locality in deep diffusion models emerges as a statistical property of the image dataset and is not due to the inductive bias of convolutional neural networks, as suggested in previous work.

Locality in Image Diffusion Models Emerges from Data Statistics

Artem Lukoianov
Massachusetts Institute of Technology
arteml@mit.edu

Chenyang Yuan
Toyota Research Institute
chenyang.yuan@tri.global

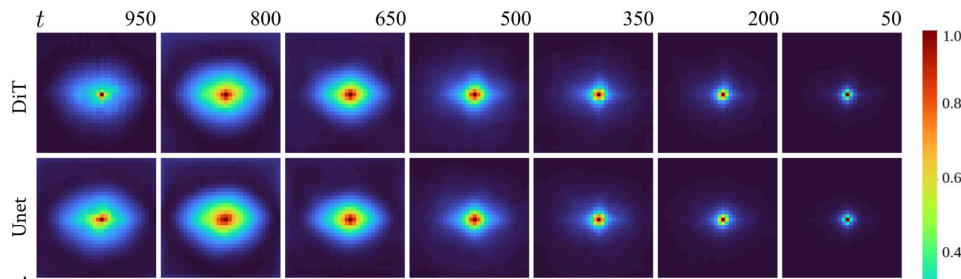
Justin Solomon
Massachusetts Institute of Technology
jsolomon@mit.edu

Vincent Sitzmann
Massachusetts Institute of Technology
sitzmann@mit.edu

<https://locality.lukoianov.com>

But wait...

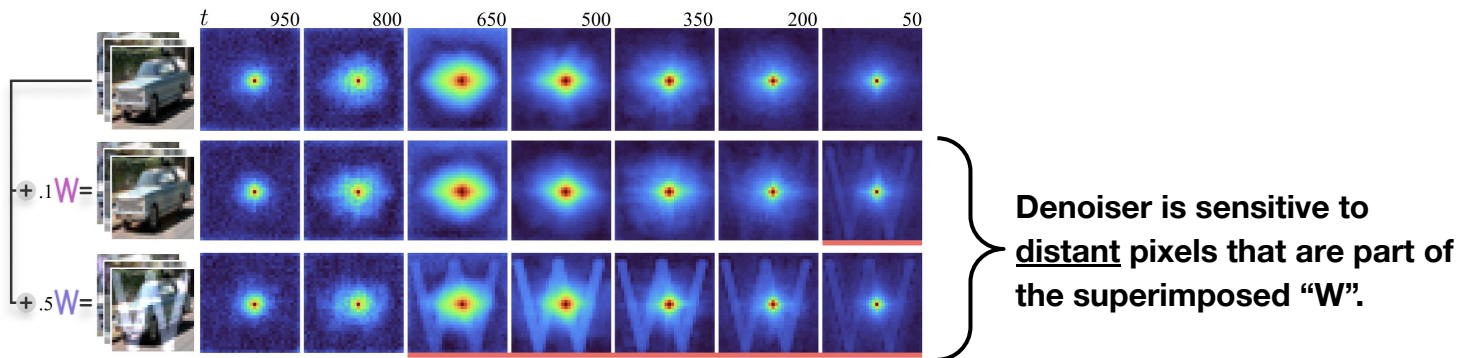
Kamb and Ganguli (2025): Convolutional architectures have a locality bias: Outputs depend on their inputs only through a small receptive field of neighboring input pixels.



Lukoianov et al. (2025): In practice, non-convolutional architectures exhibit *the same behavior*!

Locality emerges from data statistics

...and we can force a ConvNet to have non-local sensitivity fields by manipulating the training data.



Suggests that locality bias comes from the structure of the *training data* rather than the architecture.

Stability and generalization

An old literature connects *stability* and
generalization.

Journal of Machine Learning Research 2 (2002) 499-526

Submitted 7/01; Published 3/02

Stability and Generalization

Olivier Bousquet
CMAP, Ecole Polytechnique
F-91128 Palaiseau, FRANCE

BOUSQUET@CMAPX.POLYTECHNIQUE.FR

André Elisseeff
BIOwulf Technologies
305 Broadway,
New-York, NY 10007

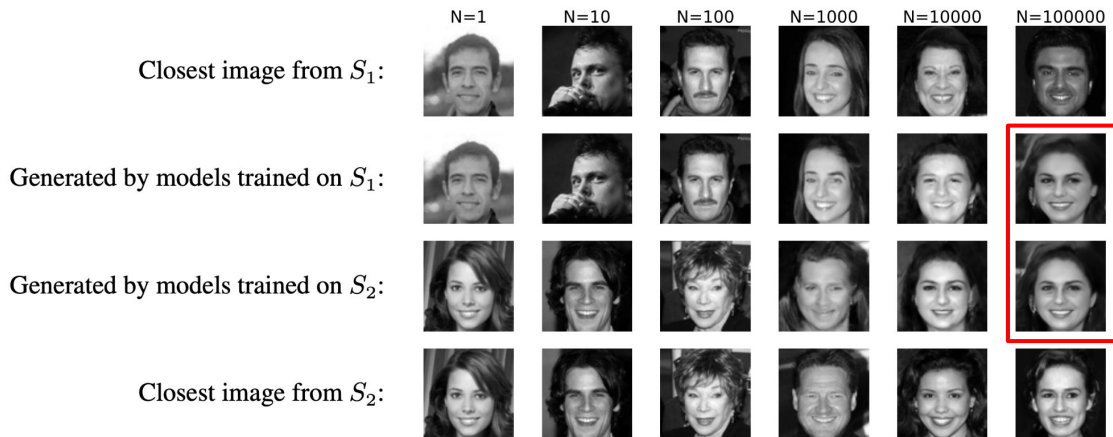
ANDRE.ELISSEEFF@BIOWULF.COM

Stability and generalization

If a model isn't too sensitive to perturbations in its training set, then it must generalize!

Intuition: Can't overfit to the training data if the model doesn't care about the training data.

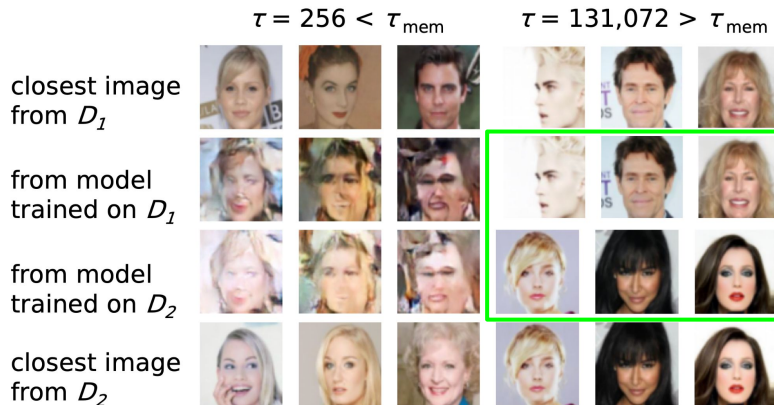
Diffusion models are stable...



Diffusion models trained on *disjoint* datasets can output nearly identical samples! ([Kadkhodaie et al. 2024](#))

...in the early stages of training

Favero et al. (2025): This phenomenon vanishes if you train for long enough.



Beyond a threshold number of training steps, diffusion models trained on disjoint datasets D_1 and D_2 diverge and memorize their training data.

Stability of SGD

This is consistent with results on the stability of models trained via *stochastic gradient descent* (SGD).

Train faster, generalize better: Stability of stochastic gradient descent

Moritz Hardt
Benjamin Recht
Yoram Singer

MRTZ@GOOGLE.COM
BRECHT@BERKELEY.EDU
SINGER@GOOGLE.COM

Train via SGD for few epochs → stability → generalization



Stability of diffusion models

Excellent theoretical analysis in
Farghly et al. (2025)!

Impact of:

- the denoising objective,
- stochastic gradient descent, and
- discrete-time sampling

on a diffusion model's stability.

Implicit Regularisation in Diffusion Models:
An Algorithm-Dependent Generalisation Analysis

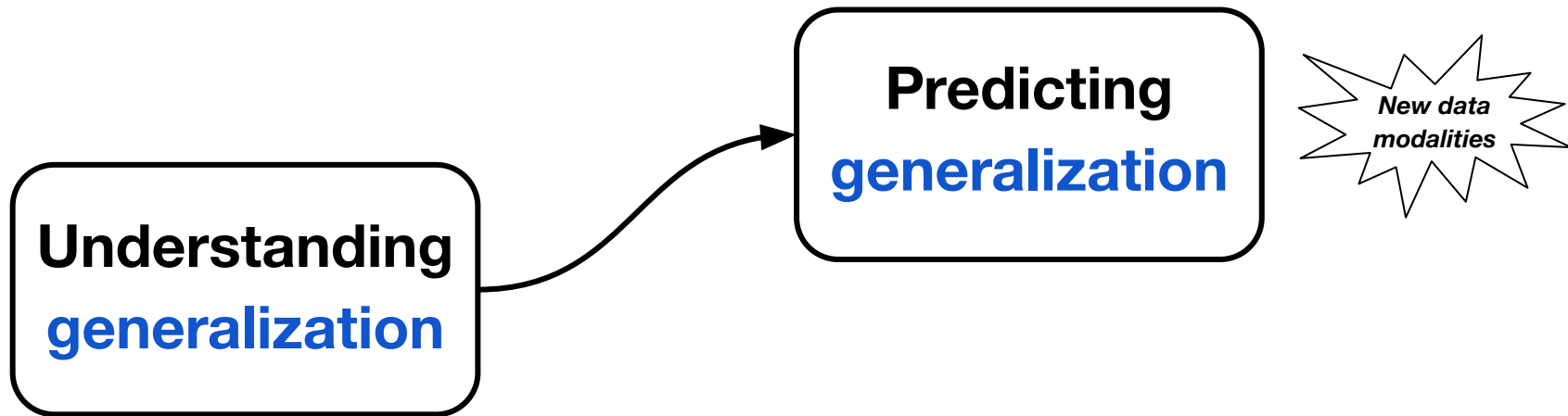
Tyler Farghly*, Patrick Rebeschini†, George Deligiannidis‡ and Arnaud Doucet§

Department of Statistics, University of Oxford

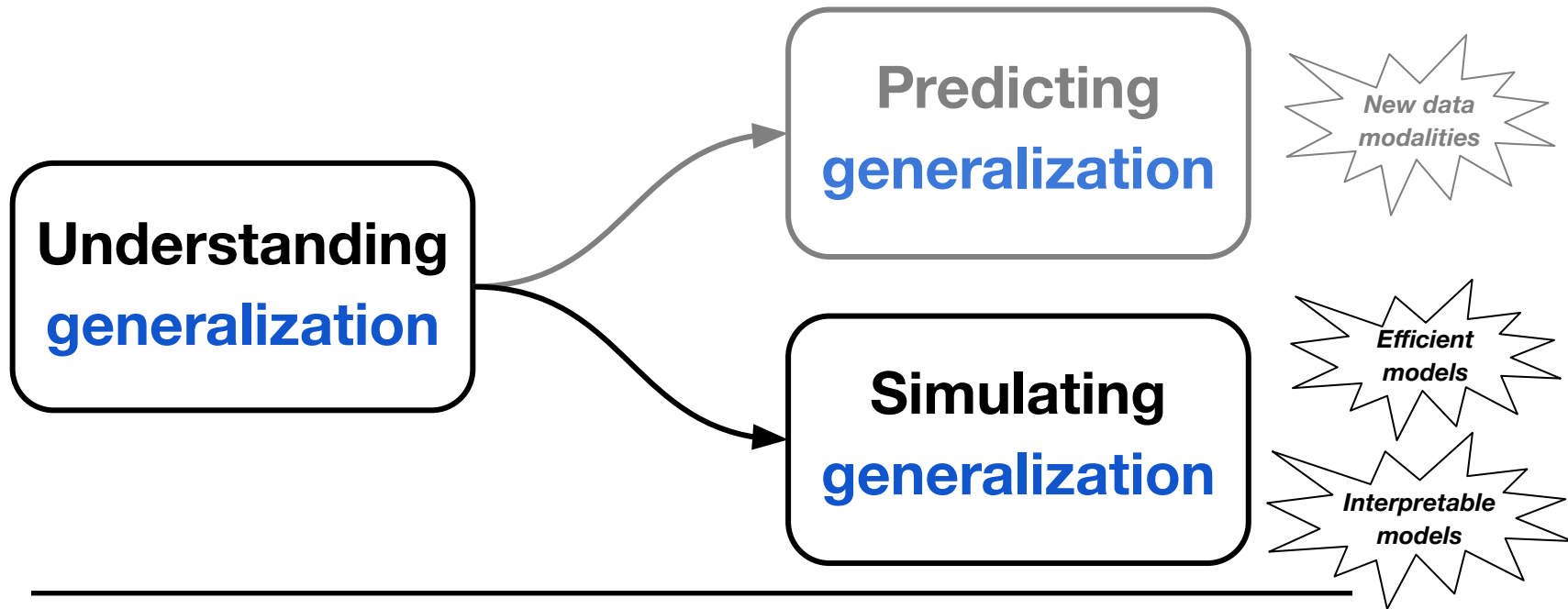
July 8, 2025



Why should practitioners care?



Why should practitioners care?



Thank you from your instructors!

Instructors



Chenyang
Yuan



Cole Becker



Artem
Lukoianov



Daniel
Pfrommer



Christopher
Scarvelis

Final lecture logistics

- Problem set #2 due tonight at 11:59 PM
- Mini-project due Friday Jan. 23 at 11:59 PM
- Lecture 6 attendance survey:

